

Proyecto de Curso: Análisis, Diseño y Construcción de un Sistema Operativo desde Cero

Por

Castro Pari, Rayneld Fidel

Mamani Flores, Natan

Mendoza Quispe, Jose Daniel

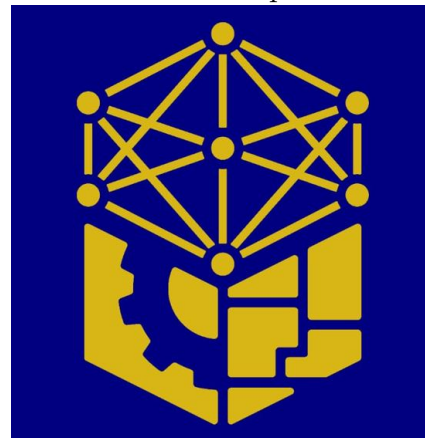
Polo Chura, Marco Rosau

Trabajo académico presentado a la

Facultad de Ingeniería Eléctrica, Electrónica, Informática y Mecánica

como

Proyecto de Investigación de la Primera Unidad de Sistemas Operativos



Departamento Académico de Ingeniería de Informática y de Sistemas

Asesor: Ugarte Rojas, Héctor Eduardo

Memorial Universidad Nacional San Antonio Abad del Cusco

Semestre 2025-II

Cusco

Perú

Índice general

Índice de tablas	iv
Índice de figuras	v
Introducción	1
1 Propuestas analizadas	4
1.1 Theseus	5
1.1.1 Nombre del proyecto o sistema operativo	5
1.1.2 Enlace al repositorio y/o documentación oficial	5
1.1.3 Objetivo del proyecto	5
1.1.4 Lenguaje(s) de implementación	5
1.1.5 Arquitectura del sistema (monolítica, microkernel, etc.)	6
1.1.6 Componentes implementados (procesos, memoria, archivos, etc.)	8
1.1.7 Herramientas utilizadas (compiladores, emuladores, etc.) . . .	8
1.1.8 Nivel de complejidad y accesibilidad para estudiantes	8
2 Comparación técnica	9

3 Análisis crítico	10
Referencias	11

Índice de tablas

1.1	Comparación entre un kernel tradicional y el nanocore de Theseus . .	7
-----	--	---

Índice de figuras

1.1	Comparativa de la arquitectura de Theseus frente a arquitecturas tradicionales.	8
-----	---	---

Introduccion

El desarrollo y la transformación de los sistemas operativos han sido fundamentales en la evolución de las tecnologías de la información. Desde los primeros programas creados para las computadoras centrales hasta los sistemas más utilizados en ordenadores personales, dispositivos móviles y equipos embebidos, los sistemas operativos han actuado como un puente entre las necesidades del usuario y la complejidad del hardware. Estudiarlos no solo permite entender el funcionamiento básico de una computadora, sino que también brinda las bases conceptuales y técnicas necesarias para el diseño de nuevas soluciones informáticas. Este documento tiene como finalidad ofrecer una perspectiva global de los sistemas operativos, abordando sus características principales, su arquitectura y sus componentes.

Objetivo de la investigacion

El objetivo principal de este trabajo es realizar un análisis técnico y comparativo de distintos sistemas operativos, con la finalidad de identificar sus características más relevantes y examinar las diversas concepciones y enfoques aplicados en su diseño e implementación. Se estudiarán tanto arquitecturas tradicionales, como la monolítica,

así como modelos más modernos, entre ellos los exokernels, híbridos y microkernels . Del mismo modo, se pretende ofrecer una visión crítica sobre las ventajas, limitaciones y ámbitos de aplicación de cada sistema operativo.

Alcance del análisis

El presente estudio se centra en sistemas operativos ampliamente utilizados y representativos de diferentes enfoques arquitectónicos y áreas de aplicación. No se abordarán sistemas altamente experimentales o versiones obsoletas a menos que sean relevantes para la comparación histórica o conceptual. El análisis incluye tanto sistemas de propósito general (Windows, macOS, Linux) como sistemas especializados (RTOS, embebidos y móviles), enfocándose en aspectos técnicos, operativos y comunitarios que permitan identificar fortalezas, limitaciones y escenarios de uso recomendados para cada sistema operativo.

Metodologia

Con el fin de alcanzar los objetivos planteados, se seguirá una metodología compuesta por tres fases fundamentales:

1. **Revisión bibliográfica y documental:** Se consultarán libros especializados, artículos académicos, blogs técnicos, documentación oficial y repositorios relacionados con los sistemas operativos en estudio.
2. **Evaluación técnica:** Se examinarán los aspectos esenciales de cada sistema, incluyendo la gestión de procesos, la administración de memoria, los sistemas de

archivos, el manejo de dispositivos, las interfaces de usuario y los mecanismos de seguridad.

3. **Comparación estructurada:** Se elaborará una tabla que sintetice las características más representativas de los sistemas operativos analizados, tomando en cuenta factores como el tamaño del kernel, la arquitectura, la comunidad, el lenguaje de implementación, la documentación disponible y el soporte de hardware.

Este enfoque permitirá garantizar un análisis más objetivo, completo y útil, tanto para fines académicos como prácticos.

Capítulo 1

Propuestas analizadas

1.1 Theseus

1.1.1 Nombre del proyecto o sistema operativo

El nombre del sistema operativo es “Theseus”; según (Theseus OS Project, 2023), fue nombrado así en honor a la paradoja del barco de Teseo, que plantea la cuestión de, si a un barco le renuevas todas sus piezas, ¿Ese barco renovado sigue siendo el mismo barco? Esta idea de renovación de cada componente es característica de este sistema operativo.

1.1.2 Enlace al repositorio y/o documentación oficial

- **Enlace al repositorio oficial:** <https://github.com/theseus-os/Theseus>
- **Enlace al libro oficial:** <https://www.theseus-os.com/Theseus/book/index.html>

1.1.3 Objetivo del proyecto

Según (Boos et al., 2020), este sistema operativo es en esencia experimental, aunque puede usarse como objeto de estudio debido a su innovadora arquitectura. También tiene objetivos técnicos, como reestructurar la modularidad, reducir el “state spill” y la recuperación ante fallos (fault recovery).

1.1.4 Lenguaje(s) de implementación

Según (Boos et al., 2020) y confirmando con el repositorio oficial (Theseus OS Project, 2024), el sistema operativo theseus fue casi completamente desarrollado en Rust, aunque también se ha usado C para una futura implementación de la librería “libc”

dirigida a Theseus (tlibc), ya que por el momento se tomó prestado de la biblioteca de REDOX (relibc).

1.1.5 Arquitectura del sistema (monolítica, microkernel, etc.)

Theseus, como sistema operativo no tiene una arquitectura clásica como monolítica o microkernel, ni mucho menos multikernel; sino que basa su estructura en **cells** o células por su traducción en español, las cuales haciendo honor a su definición biológica, son módulos pequeños e independientes que sirven como bloque de construcción para el sistema operativo. Adicionalmente, theseus tiene un componente mínimo el cual es llamado “nano-core”, que se encarga de iniciar el sistema, establecer memoria mínima y cargar las demás células; pero a diferencia de un kernel propiamente dicho, este no es un jefe permanente, sino que cada cell es independiente (Boos et al., 2020). A continuación una tabla comparativa entre un kernel clásico y el nano-core de theseus:

Tabla 1.1: Comparación entre un kernel tradicional y el nanocore de Theseus

Aspecto	Kernel tradicional	Nanocore (Theseus)
Definición	Es el núcleo del sistema operativo. Centraliza la gestión de componentes y recursos.	Unidad mínima del núcleo que gestiona sólo lo esencial para iniciar y mantener cells.
Estructura	Monolítica o microkernel: el kernel controla los servicios del sistema.	No existe un “núcleo único”: el sistema está compuesto por módulos cooperativos.
Acoplamiento	Alto. Los módulos dependen del kernel y de otros subsistemas.	Bajo. Cada módulo es reemplazable y se comunica mediante interfaces explícitas.
Objetivo	Proveer servicios centrales de manera estable.	Eliminar el <i>state spill</i> (derrame de estado), permite reemplazar componentes en ejecución.

Fuente: Adaptado en base a (Boos et al., 2020), *Theseus: An Experiment in OS Structure for State Management*.

Para entenderlo mejor, se especifica que se impone una arquitectura/organización plana para estas células, todo se corre en un solo SAS (single address space) y un solo nivel de privilegio (no se diferencia entre modo usuario y modo kernel) (Boos and Zhong, 2017). Además, se incluye una figura comparativa entre arquitecturas clásicas y la de theseus basada en células:

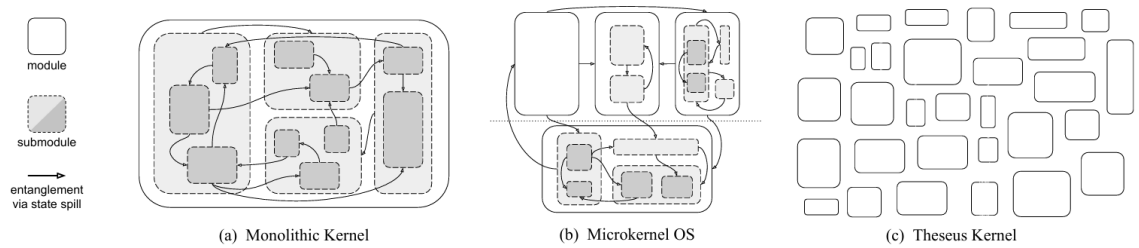


Figura 1.1: Comparativa de la arquitectura de Theseus frente a arquitecturas tradicionales.

Fuente: Obtenido de (Boos and Zhong, 2017) *Theseus: State Spill and OS Structure*.

1.1.6 Componentes implementados (procesos, memoria, archivos, etc.)

1.1.7 Herramientas utilizadas (compiladores, emuladores, etc.)

1.1.8 Nivel de complejidad y accesibilidad para estudiantes

Considerando un entorno académico, que es justamente donde rust brilla, es un proyecto digno de estudio, con un alto nivel de complejidad debido sobre todo a su arquitectura muntikernel; también cabe mencionar que la dificultad sube debido al uso de (sus herramientas, mencionar) Sin embargo, se ve compensado debido a su alta accesibilidad, la cual es posible gracias a su extensa documentación y estudios en torno a este proyecto.

Capítulo 2

Comparación técnica

La tabla siguiente presenta una comparación técnica de varios sistemas operativos destacados, considerando aspectos clave como arquitectura, lenguaje de programación, tamaño del kernel, soporte de hardware, documentación y comunidad de usuarios y desarrolladores.

Capítulo 3

Análisis crítico

Referencias

Boos, K., Liyanage, N., Ijaz, R., and Zhong, L. (2020). Theseus: an experiment in operating system structure and state management. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 1–19. USENIX Association.

Boos, K. and Zhong, L. (2017). Theseus: a state spill-free operating system. In *Proceedings of the ACM Workshop on Programming Languages and Operating Systems (PLOS '17)*. ACM.

Theseus OS Project (2023). Theseus documentation: Introduction. Recuperado de <https://www.theseus-os.com/Theseus/book/index.html>.

Theseus OS Project (2024). theseus-os/theseus (commit 1fbfe567075a65ed749b6680db1aeb538819c70c) [código fuente]. <https://github.com/theseus-os/Theseus/tree/1fbfe567075a65ed749b6680db1aeb538819c70c>. Visitado el día DD Mes 2025.